Procesamiento de Datos Masivos

Clase 04 - Data Warehousing

Objetivos

- Entender por qué son necesarias las bases de datos analíticas
- Fundamentos del Data Warehousing
- Cómo podemos mover los datos entre entornos
- Entender cómo usar BiqQuery y cómo funciona

1. OLTP vs OLAP: ¿Para qué bases de datos analíticas?

Online Transaction Processing (OLTP)

- Pensadas para responder muchas **consultas pequeñas** de forma rápida y concurrente
- Son las bases de datos que suelen verse en sistemas en producción
- Entender las consultas que se harán y un correcto modelado permiten realizar optimizaciones
- Con los índices, tenemos cómo consultar a estos datos y modelos **aprovechando el uso de memoria**

- Tenemos nuestra aplicación en producción
- Queremos estudiar los datos que tenemos
- Veamos un ejemplo simple: total de productos vendidos por mes

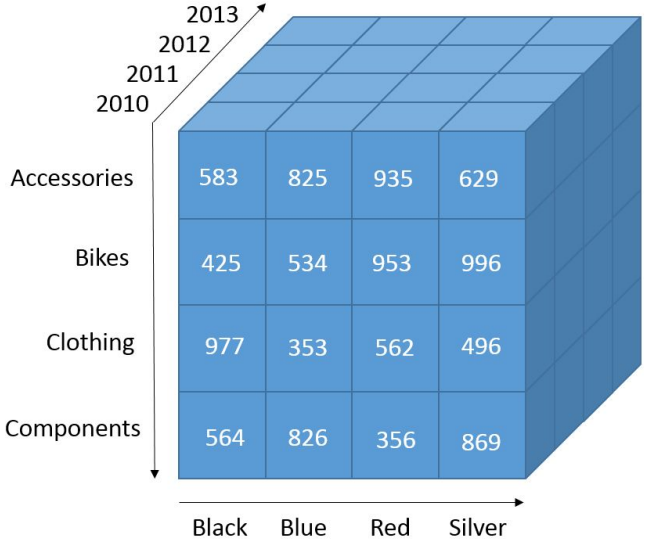
```sql
-- Total items sold per product and month
SELECT
    p.name as product_name,
    DATE_FORMAT(o.date, '%Y-%m') as month_year,
    SUM(o.total_items) as total_sales
FROM order o
JOIN product p ON o.id_product = p.id
GROUP BY p.name, DATE_FORMAT(o.date, '%Y-%m')
```

- La consulta requiere scans completos de las tablas
- Usamos muchos recursos, bloqueamos tablas y usuarios podrían verse afectados
- Esta es una consulta *simple*, pero si necesitamos cruzar más información con operaciones más pesadas...

```sql
-- Total items sold per product and month
SELECT
    p.name as product_name,
    DATE_FORMAT(o.date, '%Y-%m') as month_year,
    SUM(o.total_items) as total_sales
FROM order o
JOIN product p ON o.id_product = p.id
GROUP BY p.name, DATE_FORMAT(o.date, '%Y-%m')
```

Online Analytical Processing (OLAP)

- Paradigma para responder consultas de analítica de un negocio considerando diferentes dimensiones (ubicación, tiempo, tipos de producto, ...)
- Se basa en idea de armar u
(o
- Estos cubos se precalculan,
responder rápidamente a l
- Pero con el trade-off de ser
una nueva dimensión requie
todo el



Relational Online Analytical Processing (ROLAP)

- El problema de rigidez se soluciona usando ROLAP
- Hacemos analítica sobre una base de datos que usa un esquema relacional, como **BigQuery**
- **"Typical BigQuery use cases include large-scale storage and analysis or online analytical processing (OLAP)"**
- Pensado para **menor frecuencia** en las consultas, pero mucho **más pesadas**

Relational Online Analytical Processing (ROLAP)

- Lógica de funcionamiento se basa en almacenar datos de forma columnar
- Así se evita cargar filas completas cuando solo se usan algunos campos
- Operaciones como SUM, y análisis sobre gra son mucho más eficientes

```sql
-- Total items sold per product and month
SELECT
    p.name as product_name,
    DATE_FORMAT(o.date, '%Y-%m') as month_year,
    SUM(o.total_items) as total_sales
FROM order o
JOIN product p ON o.id_product = p.id
GROUP BY p.name, DATE_FORMAT(o.date, '%Y-%m')
```

Cómo almacenamos la información

Data Warehouse

- Si vamos a hacer analítica, necesitaremos cruzar información de distintas fuentes: DBs, APIs, CRMs...

- Un **Data Warehouse** es un tipo de sistema de datos que está diseñado para habilitar y apoyar en tareas de Business Intelligence (BI), especialmente el análisis de datos

- Debe ser un espacio que **centralice** la información ya **procesada** como fuente de verdad para los analistas

Data Warehouse Relacional

- En general, en un Data Warehouse relacional, voy a disponibilizar tablas que no están normalizadas para facilitar la analítica[1]

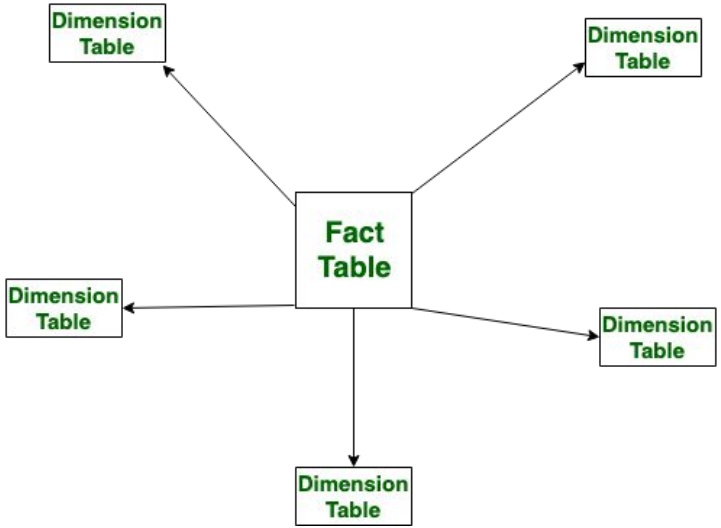
- Los dos tipos de modelamiento más clásico en este contexto son **Star Schema** y **Snowflake Schema**

- Ambos se basan en el uso de
  - **Fact tables**: qué ocurrió
  - **Dimensions:** contexto de qué, quién, cuándo...

1: Podemos relajar el uso de Formas Normales
2: No se usan índices en estos sistemas, ya veremos por qué

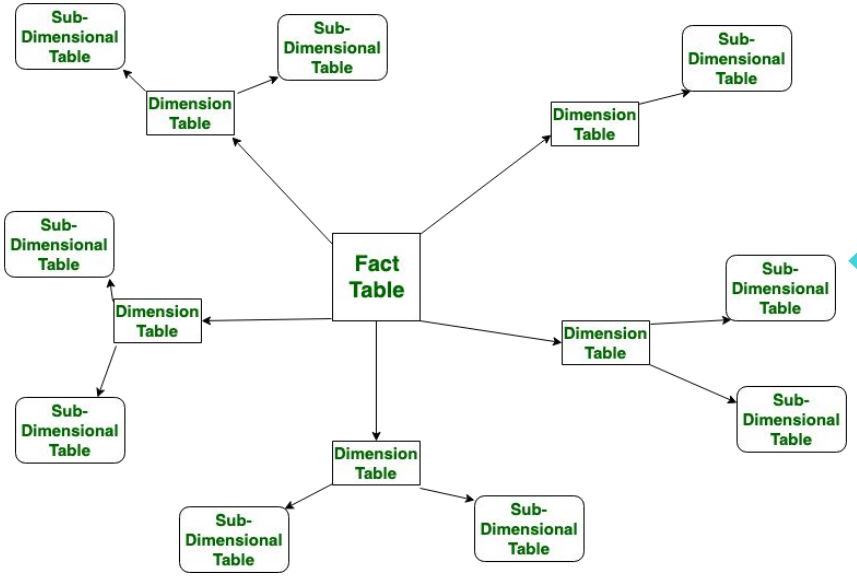
Star Schema

- Esquema más desnormalizado
- Tenemos la tabla de hechos central rodeada de dimensiones



Snowflake Schema

- Esquema busca normalización
- Tabla de hechos central, rodeada de dimensiones y estas dimensiones suelen estar normalizadas



Star vs Snowflake schema

Star Schema:

- Es la opción más común en Data Warehouses.
- Consultas más simples y rápidas (menos joins).
- La redundancia no suele ser problema debido al bajo costo de almacenamiento.

Snowflake Schema:

- Principalmente cuando existen dimensiones muy grandes o complejas.

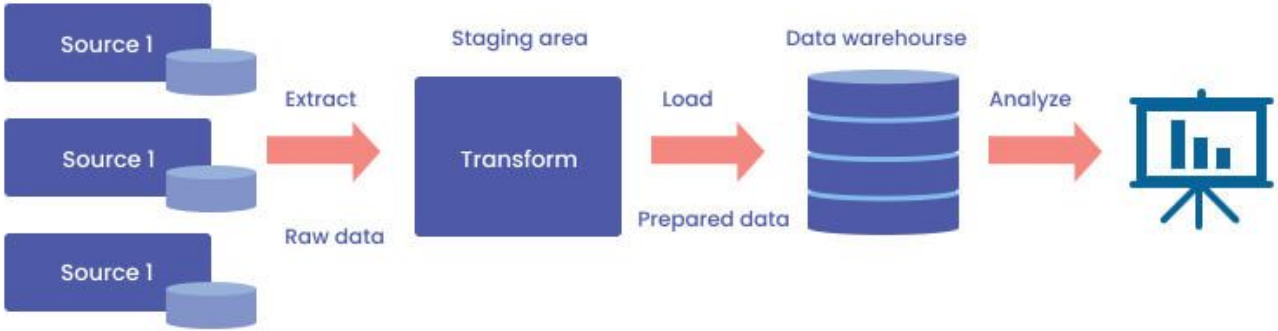
Recapitulando

- No usar OLTP en producción para hacer analítica
- Los Warehouse nos permiten tener un espacio con datos centralizados y limpios para realizar analítica
- Usando ROLAP tenemos un sistema centralizado, flexible y optimizado para consultas complejas

¿Cómo llevamos los datos a un Data Warehouse?

Data Pipelines

- Infraestructura que se va a encargar de mover los datos desde su fuente hasta el Data Warehouse
- Principales enfoques: Extract-Transform-Load (ETL) y ELT



ETL vs ELT

- Cuando el costo de almacenamiento era mayor, no se quería tener “datos sucios”. Por lo que antes de almacenarlos, se limpiaban

- Los costos actuales nos permiten tener una capa intermedia entre el origen y el warehouse: los **Data Lake**

- Estos son usados para almacenar los datos crudos sin estructuras fijas, teniendo mayor flexibilidad y guardando datos *“por si acaso”*

Stack de tecnologías

Un stack moderno para construir un pipeline de datos incluye las siguientes tecnologías:

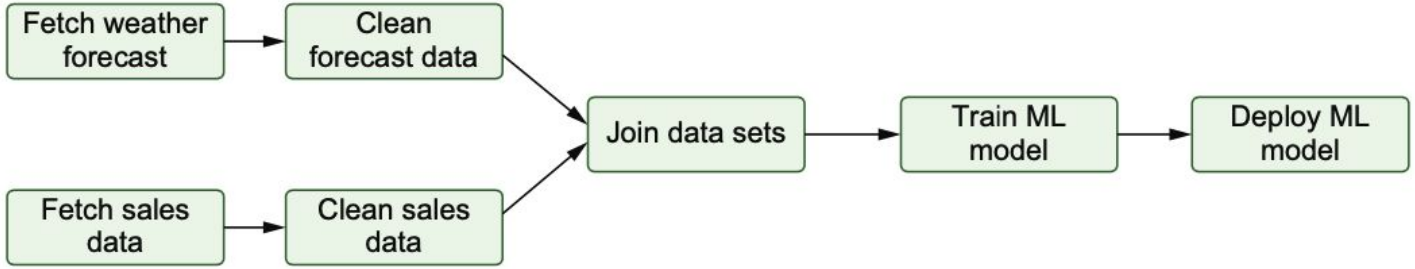
- Orquestador de jobs, como Airflow
- Docker y Kubernetes
- Herramienta para armar las vistas, como DBT
- Herramientas en la nube, como GCS, BigQuery, Cloud Composer...

Airflow

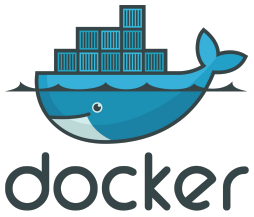
- Apache Airflow es una herramienta para orquestar tareas
- Estas tareas son encapsuladas en operadores (ej. PythonOperator, BashOperator, KubernetesPodOperator)
- Algunas personas lo definen como "cron on steroids"

Airflow

Se basa en la idea de definir un DAG (grafo dirigido acíclico) de tareas; este DAG se define mediante código Python



Docker



- Docker es una herramienta que nos permite encapsular código en imágenes, y ejecutarlas como contenedores
- Una imagen es un "computador base" que está listo para correr un comando
- Nos permite tener un entorno de ejecución uniforme en todos los lugares en que el código corra

Docker

```
FROM python:alpine

# Install GCC
RUN apk add gcc musl-dev libffi-dev

# Upgrade pip
RUN pip install --no-cache-dir --upgrade pip

# Install poetry
RUN pip install poetry

# Installing dependencies
RUN mkdir /app
WORKDIR /app

COPY pyproject.toml /app/

RUN poetry install --no-dev

COPY . /app

CMD ["poetry", "run", "python", "/app/src/main.py"]
```

Kubernetes

- Kubernetes, also known as K8s, is an open-source system for automating deployment, **scaling, and management of containerized applications**

- Kubernetes es un tópico en sí mismo, pero estudiemos las nociones necesarias para construir un pipeline de datos

- La idea a grandes rasgos es que vamos a tener un cluster de Kubernetes que va a tener los recursos computacionales necesarios para correr nuestras imágenes

- Airflow va a estar montado en un cluster de Kubernetes

DBT

- DBT es una herramienta que nos permite transformar datos dentro del Data Warehouse usando SQL y buenas prácticas

- Un modelo es una consulta SQL que transforma datos y genera una tabla o vista nueva en el warehouse.

- Nos permite organizar, versionar, testear y documentar transformaciones de datos de forma reproducible.

Arquitectura del pipeline

- El código de procesamiento de datos se empaqueta en contenedores con Docker
- Estos contenedores se ejecutan y escalan en la infraestructura usando Kubernetes
- Airflow orquesta el pipeline, definiendo el orden y la programación de las tareas
- Dentro de esas tareas dbt se utiliza para transformar los datos en el Data Warehouse mediante modelos SQL que limpian y organizan la información para su análisis

Recapitulando

- Entendemos por qué queremos llevar nuestros datos a un Warehouse
- Vimos una arquitectura que nos permite llevar nuestros datos hasta donde podremos analizarlos
- Nos falta el core del stack: el warehouse

Nuestro Warehouse: BigQuery

Data Warehousing en la nube

- Ya sabemos que un Data Warehouse es el lugar donde guardamos información procesada, lista para que los analistas puedan consumirla

- Un Data Warehouse moderno se construye sobre un sistema en la nube como BigQuery, Snowflake o Redshift

- En este curso vamos a aprender a usar BigQuery, pero todos estos sistemas tienen los mismos principios

BigQuery

*BigQuery is Google's fully managed, **serverless data warehouse** that enables scalable analysis over petabytes of data*

Algunas claves:

- **Serverless**
- **Auto-escalable**
- Pay as you use

BigQuery

*Serverless computing is a **cloud computing execution** model in which the cloud provider **allocates machine resources on demand**, taking care of the servers on behalf of their customers*

- El proveedor escala horizontalmente en la medida que necesitemos más recursos
- Nosotros interactuamos con una interfaz de alto nivel
- Tenemos espacio y cómputo “ilimitado” (en la medida que podamos pagar)
- Tenemos integraciones con diversos lenguajes/programas

Warehouse (arquitectura)

La arquitectura de un sistema de Data Warehouse en la nube se basa en un almacenamiento distribuido

Al hacer consultas comunicamos datos a un cluster de cómputo de alta disponibilidad

Al hacer una consulta disponemos de **muchos** "computadores pequeños" (slots) para responder

Algoritmos en BigQuery

- BigQuery divide una consulta en varias consultas pequeñas
- Cada sub-consulta se envía a un slot de trabajo distinto
- En general, las consultas hacen hashing a los distintos servidores

Broadcast Join (de la documentación de BQ)

- When joining a large table to a small table, BigQuery creates a broadcast join where the small table is sent to each slot processing the large table

- Even though the SQL query optimizer can determine which table should be on which side of the join, it is recommended to order joined tables appropriately

Hash Join (de la documentación de BQ)

- When joining two large tables, BigQuery uses hash and shuffle operations to shuffle the left and right tables so that the matching keys end up in the same slot to perform a local join. This is an expensive operation since the data needs to be moved.

Particiones y Clustering

BigQuery **no usa índices** tradicionales. Se emplea almacenamiento **columnar** junto con **particionamiento** y **clustering** para optimizar el acceso a los datos.

Por lo tanto, los algoritmos de consulta están diseñados para procesar grandes volúmenes de datos **en paralelo**, reduciendo la cantidad de datos escaneados

Particiones y Clustering

Partitioning: técnica que permite escanear parte de los datos (segmentos o particiones), no toda la tabla

En general se aplica sobre campos de tipo fecha/timestamp

Es clave para ahorrar dinero ($) a la hora de hacer consultas sobre datos masivos

Particiones y Clustering

Clustering: Permite organizar los datos dentro de cada partición, ordenando por una o varias columnas

Se usa en columnas de alta cardinalidad (como country, status, user_id, etc.), permitiendo mayor eficiencia a la hora de aplicar los filtros

Particiones y Clustering

En resumen:

- **Partitioning**: reduce el tamaño del problema (qué bloques leer)
- **Clustering**: optimiza la búsqueda dentro de esos bloques

BigQuery y SQL

- BigQuery tiene algunos comandos específicos, pero en general es igual al SQL de toda la vida

- Además de hacer joins y agregaciones, existe un tipo de comandos muy importante en Data Warehousing: las **Window Functions**

- Estudiaremos estas funciones con algunos ejemplos prácticos

Antipatrones

- Uso de self-joins y cross-joins. Mejor buscar usar solo equi-joins y uso de Window Functions
- ORDER BY: requiere materializar todos los resultados en un nodo, por lo mismo para resultados grandes no se recomienda
- SELECT * : Es el pecado capital en BQ. Perdemos toda la gracia de que sea una BDD columnar

Anexo

- Si quieren saber más de BQ, les sugiero investigar sobre **Dremel** y **Colossus**

- Parte de los contenidos que vimos hoy han estado cambiando en los últimos años. Un tipo de storage llamado ***lakehouse*** permite unir la flexibilidad y bajo costo de las tecnologías vistas. Investigar ***Delta Lake*** o ***BigLake***